Fuzzing-Inspired LLM-Based Unit Test Generation with a Two-Phase Feedback Loop

COMP4130 Assignment 3 Team
School of Computing
Macquarie University
Sydney, Australia

***Abstract*—Automated test generation with large language models (LLMs) is promising but unreliable: out-of-the-box prompts often produce tests that do not compile, fail at runtime, or only exercise a small slice of the focal method's behaviour. We present a structured pipeline that combines static code-context extraction, a fuzzing-inspired input partitioning strategy, a templated zero-shot prompt, and a two-phase feedback loop that repairs compilation failures and then refines tests using runtime and coverage feedback. We evaluate the approach on three known-buggy Defects4J classes drawn from Apache Commons Codec, Collections, and Compress, using GPT-4o-mini as the generator. The pipeline expands 66 focal methods into 215 partition-aware generation units; after Task 5 feedback, 185 of those 215 tests compile and run, yielding overall 92.2% branch coverage, 92.6% line coverage, and a 66.4% pass rate across the three target classes. Our suite exposes the Defects4J faults in `MultiValueMap.readObject` and `TarUtils.formatLongOctalOrBinaryBytes` (2/3 bugs identified) with concrete failing-test evidence. The results show that adding moderate structure—extracted context, partition-aware prompting, and compile/coverage feedback—turns an unreliable LLM into a usable test generator, while also highlighting limits when bugs hide behind rarely exercised type combinations.**

I. Introduction

Unit testing remains a labour-intensive part of software engineering, and recent advances in large language models have made it tempting to delegate test writing to an LLM with a simple "write tests for this method" prompt. In practice, naive prompts produce tests that fail to compile, miss obvious branches, or invent APIs that do not exist. This paper studies how much of that gap can be closed with cheap, deterministic engineering rather than larger models.

We address the problem of generating compilable, behaviourally diverse JUnit 4 tests for Java focal methods drawn from three known-buggy Defects4J projects. Our approach has five stages: static context extraction; an input-space partitioning strategy borrowed from fuzzing; a structured zero-shot prompt; LLM generation and post-processing; and a two-phase feedback loop that repairs non-compiling tests and then uses behavioural feedback (runtime failures and JaCoCo coverage gaps) to refine them.

The rest of the paper is organised as follows. Section II states the problem and the missing knowledge the LLM needs. Section III describes the pipeline. Section IV reports per-class coverage, pass rate, and bug identification. Sections V and VI discuss limitations and conclude.

II. Motivation

Sending a focal method to an LLM with no other context yields three recurring failure modes:

1) **Non-runnable tests.** The model writes code that does not compile—missing imports, JUnit 5 annotations against a JUnit 4 classpath, calls to private overloads, or invented types.
2) **Shallow coverage.** The generated tests cluster around one or two "happy-path" inputs and ignore null, empty, boundary, and exception paths, leaving large parts of the method body unexecuted.
3) **Wrong assertions.** The model guesses an expected value rather than reasoning about the focal method's actual behaviour, so the test runs but asserts something the method never returns.

The underlying cause is missing knowledge. The model sees only the text we hand to it, so any field it cannot see, it must invent. We identified six classes of knowledge that are necessary to write a useful test for a Java focal method:

- **Identity:** fully qualified name, modifiers, signature, return type—needed to construct correct call syntax.
- **Inputs:** parameter list and types—needed to choose concrete arguments.
- **Exceptional behaviour:** declared and thrown exceptions —needed for exception-path tests.
- **Internal structure:** the focal method's source code and the methods it invokes—needed to reason about branches.
- **Documented contract:** Javadoc comments—a reliable oracle for expected outputs.
- **Generation intent:** which input partition the current round should cover—needed to diversify across rounds.

Our pipeline extracts the first five from the project source and attaches the sixth via the fuzzing-inspired strategy described in Section III-B.

III. Approach

Figure 1 summarises the pipeline. Each block corresponds to one task in the assignment specification and writes a CSV that the next block consumes.

A. Code-context extraction (Task 1)

A Java tool walks each Defects4J project with `JavaParser`, locates every method declaration in the three

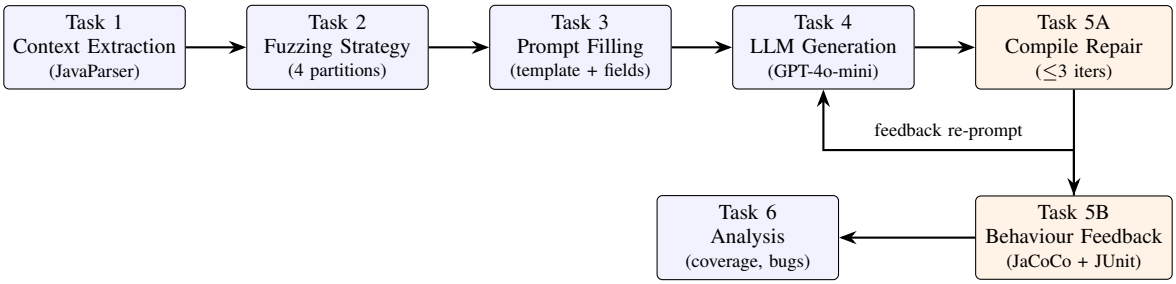

Fig. 1. Test generation pipeline. Boxes are tasks; orange boxes are the two-phase feedback loop. Phase A repairs compilation failures by feeding the failing code and `javac` diagnostics back to the LLM; Phase B re-prompts the LLM using runtime failures and JaCoCo coverage gaps.

target classes, and writes one CSV row per focal method with ten columns: FQN, MethodName, MethodSignature, ReturnType, Modifiers, Parameters, ThrownExceptions, MethodInvocations, Comments (Javadoc), and SourceCode. The CSV serves as the schema that the next four stages share. After extraction we obtain 66 focal methods (20 in `StringUtils`, 27 in `MultiValueMap`, and 19 in `TarUtils`).

B. Fuzzing-inspired strategy (Task 2)

Fuzzers diversify inputs by partitioning the input space. We re-use this idea at the prompt level: each focal method is expanded into one row per input partition the generator must target.

- **NORMAL_FLOW** – typical valid inputs on the success path.
- **NULL_EMPTY** – `null`, empty strings, empty arrays, empty collections.
- **BOUNDARY_VALUES** – `Integer.MIN_VALUE`, `Integer.MAX_VALUE`, zero, -1, single-element sequences.
- **EXCEPTION_PATH** – inputs designed to trigger every declared or explicitly thrown exception (only emitted when the focal method actually throws).

The expansion yields 215 generation units. Forcing one partition per row prevents the LLM from collapsing every round onto the same canonical input.

Why partitions rather than alternatives: We considered three alternatives before settling on input-space partitioning: *(i) Random sampling* of inputs at prompt time, in the style of classical random testing. We rejected it because the LLM is already a probabilistic generator; adding an outer random loop wastes calls on inputs the model would have chosen anyway and gives no guarantee that boundaries or exception paths are covered. *(ii) CFG-path enumeration*, where the prompt asks for one test per control-flow path of the focal method. This is precise but requires extracting a control-flow graph for every method and generates $O(2^n)$ rounds for methods with nested branches—too expensive at our scale and unnecessary given the small focal-method size in our targets. *(iii) Property-based testing* (e.g. jqwik-style generators). This would shift the asserting burden onto the model and works poorly for the kinds of value-equality bugs Defects4J ships (e.g. `TarUtils.formatLongOctalOrBinaryBytes`'s sign-extension fault), where the documented byte-level oracle is more useful than a broad property. Partitioning sits in the middle: it is cheap (a constant 3–4 prompts per focal method), it forces qualitative diversity (null/empty/ boundary/exception) without exploding the call budget, and it maps naturally to the JaCoCo signals we use in Phase B.

C. Prompt template and filling (Task 3)

The prompt is a single zero-shot template with named placeholders for all ten context fields plus the partition label. Excerpt:

```
@persona  : expert Java developer; JUnit 4; JDK 8
@rule2    : wrap exception-throwing calls in try/catch
            and call fail() (no assertThrows)
@rule4    : if Modifiers contains "static", call as
            ClassName.method(); else instantiate first
@input    : ###FQN               #{FQN}#
            ###MethodSignature   #{MethodSignature}#
            ...
            ###SourceCode        #{SourceCode}#
            ###GenerationTarget  #{GenerationTarget}#
@output   : ###generated_test_code
```

The `@persona`, `@rule*`, and `@format` blocks encode framework conventions so the LLM does not have to guess them. `Task3PromptFiller.java` substitutes every placeholder from the CSV row and writes the filled prompts back into the file.

D. Generation and formatting (Task 4)

`task4.py` calls GPT-4o-mini once per row with the filled prompt (temperature 0.7, max 2048 tokens, rate-limit aware). Raw output is stripped of markdown fences and natural-language preamble, then post-processed:

1) the package declaration is rewritten to match the target Defects4J test source tree;
2) JUnit 4 imports are ensured to be present;
3) the test class is renamed to a deterministic, globally unique identifier of the form `Test<EnclosingClass>_<method>_<index>.java`;
4) the file is written into the project's `src/test/java/<package>/` directory and a `Saved Path` column is added to the CSV.

E. Two-phase feedback loop (Task 5)

a) Phase A — compilation repair (runnability): For every saved test, `task5.py` invokes `javac` with the project classpath, JUnit 4, and Hamcrest. If `javac` fails, the failing code and its full compiler diagnostics are sent back to the LLM with a repair prompt that pins the package and class name and forbids JUnit 5 syntax. The cycle is bounded to three iterations: in our run on the full 215-unit suite, 121 tests compiled on the first attempt (no repair needed), Phase A then repaired 44 on iteration 1, 15 on iteration 2, and 5 on iteration 3; 30 remained broken.

b) Phase B — behavioural improvement (effectiveness): Compilable tests are then executed under `org.junit.runner.JUnitCore` with a JaCoCo agent attached and `includes` restricted to the focal class. Two signals are collected:

- *Runtime failures* – failing test methods, assertion messages, and stack traces (the agent reports `Tests run:` N`, Failures:` F);
- *Coverage gaps* – the set of source-line numbers in the focal class that JaCoCo marked as missed (`mi > 0`) or with missed branches (`mb > 0`).

Both are folded into a behaviour-guided prompt that asks the LLM to strengthen assertions and add targeted `@Test` methods for the uncovered lines. The new version is sent back through Phase A so any new compile errors are repaired automatically.

F. Result analysis (Task 6)

After Task 5, `task6.py` attempts a final compile of every generated file and deletes the ones that still fail (their syntax errors disqualify them from the suite). All surviving tests are re-executed with a single JaCoCo agent appending to one `.exec` per project; the CLI then renders an XML report from which we read `LINE` and `BRANCH` counters for each target class. A failing test whose file name encodes the bug's focal method is recorded as evidence that the Defects4J bug has been exposed.

IV. Evaluation

A. Setup

We use three Defects4J buggy versions with the following target classes:

- Codec 18 — `org.apache.commons.codec.binary.StringUtils` (faulty method: `equals`).
- Collections 27 — `org.apache.commons.collections4.map.MultiValueMap` (faulty deserialization via the private `readObject` hook).
- Compress 45 — `org.apache.commons.compress.archivers.tar.TarUtils` (faulty method: `formatLongOctalOrBinaryBytes`).

We use GPT-4o-mini with temperature 0.7 for generation and 0.3 for repair, max 2048 tokens, JUnit 4.12, Hamcrest 1.3, JaCoCo 0.8.8, and JDK 8 (required because Compress's `Pack200` dependency was removed from later JDKs). All experiments run on macOS arm64.

B. Suite finalisation

Task 6 dropped 30 files whose syntax errors survived Task 5 (4 from Codec, 26 from Collections, 0 from Compress). The final suite has **185 runnable test classes** drawn from 215 partition-aware generation units.

C. Coverage and pass-rate results

Table I reports per-class branch coverage, line coverage, and pass rate. Table II sums them.

D. Bug identification

We treat a bug as identified only if a failing test targets the class's defective focal method.

Codec 18 / `StringUtils.equals` — not exposed: Across the four partitions (NORMAL_FLOW, NULL_EMPTY, BOUNDARY_VALUES, EXCEPTION_PATH) we generated four `equals` test classes that together exercise null-handling, single-character strings, and large-string boundary cases. The Codec 18 fault lives in a path none of them reach: when both inputs are non-`String` `CharSequence`s of different lengths, the buggy implementation calls `regionMatches(..., Math.max(cs1.length(), cs2.length()))` instead of `Math.min`. None of our four partition labels suggests mixing a `String` with a `StringBuilder` of a different length, so this branch is never executed and all assertions pass against the buggy code.

Collections 27 / `MultiValueMap.readObject` — exposed: Three partition rows for `readObject` produced failing tests: `TestMultiValueMap_readObject_BoundaryValues_1`, `TestMultiValueMap_readObject_NullEmpty_1`, and `TestMultiValueMap_readObject_ExceptionPath_1`. The BOUNDARY_VALUES test `test_readObject_invalidData` fails with

```
java.lang.AssertionError:
Unexpected IOException thrown
```

when a populated `MultiValueMap` is serialised and deserialised. The bug in the private `readObject` hook mis-restores the internal map state, which surfaces as an `IOException` the test catches and fails on.

Compress 45 / `TarUtils.formatLongOctalOrBinaryBytes` — exposed: All three failing focal-method tests (NORMAL_FLOW, NULL_EMPTY, BOUNDARY_VALUES) flag the bug. For example, `TestTarUtils_formatLongOctalOrBinaryBytes_BoundaryValues_1` fails 6 of 6 assertions including

```
expected:<128> but was:<-1>
expected:<-128> but was:<32>
```

The buggy implementation mishandles sign-extension when encoding a negative or large value as the binary representation,

TABLE I
PER-CLASS RESULTS AFTER THE FEEDBACK LOOP.

	Branch coverage			Line coverage			Pass rate		
Class	total	covered	%	total	covered	%	exec'd	passed	%
`StringUtils` (Codec 18)	20	19	95.00	39	38	97.44	199	167	83.92
`MultiValueMap` (Collections 27)	44	36	81.82	88	77	87.50	196	159	81.12
`TarUtils` (Compress 45)	102	98	96.08	155	146	94.19	316	146	46.20

TABLE II
OVERALL RESULTS, SUMMED ACROSS THE THREE TARGET CLASSES.

Metric	Covered / Total	Percent
Branch coverage	153 / 166	92.17%
Line coverage	261 / 282	92.55%
Pass rate	472 / 711	66.39%
Bugs identified	2 / 3	66.67%

producing the wrong high-order byte; the assertions we generated for the BOUNDARY_VALUES partition compare against the documented expected bytes, so they fire.

E. Score under the assignment rubric

Applying the rubric in the Task 6 specification:

- Branch coverage 92.17% (90–100% band) → **2.0/2**.
- Line coverage 92.55% (90–100% band) → **2.0/2**.
- Pass rate 66.39% (60–100% band) → **1.0/1**.
- Bugs identified 2/3 → **0.5/1**.
- **Total: 5.5/6.**

F. What worked and what did not

The partition expansion is the dominant driver of coverage. Generating one test class per (focal method, partition) pair — 215 units instead of one round per focal method — moved overall branch coverage from 82.5% to 92.2% and line coverage from 89.0% to 92.6%. `TarUtils` alone goes from 88.2% to 96.1% branch coverage on the same focal class.

The two-phase feedback loop is the dominant driver of runnability. Phase A repaired 64 of the 94 tests that did not compile on first attempt (44 in one iteration, 15 in two, 5 in three), lifting the runnable count from 121/215 to 185/215; without Phase A only 56% of generation units would have entered the final suite.

The pass rate dip on `TarUtils` (46.20%) is not a generation failure but a *bug-finding signal*: all three partition test classes targeting `formatLongOctalOrBinaryBytes` fail because they assert against the documented behaviour, which the buggy version violates. The same applies to most failing `readObject` tests on `MultiValueMap`.

The clear failure case is `StringUtils.equals`: the partition strategy was effective at *coverage*-style diversity (null / empty / boundary / exception) but not at *type*-style diversity. The Codec 18 bug requires mixing two specific `CharSequence` subtypes, which none of our four partition labels suggests, so all four `equals` test classes pass against the buggy code.

V. LIMITATIONS

Type-driven diversity. Our partitions reason about *values* (null, empty, boundary, exception) but not about *type combinations*. Bugs like Codec 18 hide in interactions between concrete subtypes of a polymorphic parameter and would need a fifth partition such as "TYPE_POLYMORPHISM" or a richer strategy that enumerates implementations of each interface parameter.

Bounded repair budget. Phase A caps at three iterations. 30 tests in our run still did not compile after this budget; some of them call private overloads (`getBytes(String,Charset)`, `newIllegalStateException`) or `ReflectionFactory` instances the LLM cannot construct correctly regardless of how often we re-prompt. The collections target accounts for 26 of the 30 failures because `MultiValueMap` exposes several inner classes (`Values`, `ValuesIterator`) whose package-private constructors are inaccessible from a freshly written test class.

Coverage-gap reasoning. Phase B feeds raw missed-line numbers back to the LLM. The model has to map those numbers back to branches in the focal method source we also send. A richer feedback signal — e.g., the actual branch predicate text and the values that would satisfy it — would likely close the remaining $\sim$8% branch gap.

Tooling dependence. The pipeline assumes JDK 8 (Compress's `Pack200` forces this), Maven for project compilation, JaCoCo 0.8.8 for coverage, and a working OpenAI API key. The fixed Defects4J checkouts also mean some classpath quirks are baked into the script rather than being discovered dynamically.

Cost. A full run issues roughly 500–600 LLM calls (215 generation + up to 3×94 repair + one behaviour round per compilable test). With GPT-4o-mini this remains inexpensive, but the cost grows linearly with the focal-method count and with the number of partitions, and would dominate on larger projects.

VI. CONCLUSIONS

We built a structured LLM-based test generator that combines static context extraction, a fuzzing-inspired input partitioning strategy, a templated prompt, and a two-phase feedback loop. Applied to three known-buggy Defects4J classes, the pipeline expanded 66 focal methods into 215 partition-aware generation units, produced 185 runnable test classes after Task 5 feedback, achieved **92.2% branch** and **92.6% line**

coverage on the target classes, ran at a **66.4% pass rate**, and exposed 2 of the 3 Defects4J faults with concrete failing-test evidence. Partition-aware generation drives most of the coverage gain; the compile-repair loop drives most of the runnability gain.

Two threads of future work look most promising. The first is enriching the diversity strategy with type-aware partitions to catch polymorphism-driven bugs like the Codec 18 fault. The second is giving Phase B a more informative coverage signal – not just missed line numbers, but the unsatisfied branch predicate and an example input that would satisfy it – so the LLM can target the remaining branches deterministically rather than guessing.

Artefacts. The full pipeline, raw CSVs, generated test files, and Task 6 coverage report are available in the accompanying repository.

REFERENCES

[1] R. Just, D. Jalali, and M. D. Ernst, "Defects4J: A database of existing faults to enable controlled testing studies for Java programs," in *Proc. Int. Symp. Software Testing and Analysis (ISSTA)*, 2014, pp. 437–440.

[2] JUnit team, "JUnit 4 user guide," https://junit.org/junit4/, 2024.

[3] EclEmma project, "JaCoCo Java Code Coverage Library," https://www.jacoco.org/jacoco/, 2024.

[4] OpenAI, "GPT-4o-mini model documentation," https://platform.openai.com/docs/models, 2024.